KNN Arena: An Interactive Game-Based Laboratory for Comparing Standard and Distance-Weighted k-Nearest Neighbour Classifiers

Aditya Bhattacharyaa
Dept. of Computer Application
Kalyani Government Engineering College
Roll: 10271025001

Arunava Choudhury
Dept. of Computer Application
Kalyani Government Engineering College
Roll: 10271025008

Tanmay Biswas
Dept. of Computer Application
Kalyani Government Engineering College
Roll: 10271025045

***Abstract*—The k-nearest neighbour (kNN) classifier is usually the first algorithm students meet in machine learning, yet the difference between uniform voting and distance-weighted voting (wkNN) remains abstract in textbooks and static plots. This paper presents *KNN Arena*, a browser-based educational game that places kNN and wkNN head-to-head on identical data. A learner guesses the hidden class of a mystery point, watches both algorithms vote, inspects every neighbour's rank, distance and weight, and solves handcrafted puzzles that force the two algorithms to disagree. The system offers three modes (Play, Sandbox and Wedge Puzzles), four procedurally generated datasets, three distance metrics, four weighting schemes and adjustable, reversible label noise. All machine-learning logic, including distances, weights, voting, dataset generation and the puzzle solver, is implemented from scratch in TypeScript, without any ML library, and runs entirely client-side using React 18 and HTML5 Canvas. The project grew out of a Python/Tkinter prototype and was ported to the web with behavioural parity checked by 19 automated tests. Original contributions include a jitter-resilient automated par solver, a disagreement heatmap, a hard-mode sampler for contested regions and a stable label-noise model.**

***Index Terms*—k-nearest neighbours, weighted kNN, gamification, interactive learning, decision boundaries, distance metrics, React, HTML5 Canvas**

I. Introduction

Instance-based learning is a core topic in every introductory machine-learning course. The kNN rule of Cover and Hart [1] classifies a query by a vote among its k closest training points, and Dudani's distance-weighted variant [2] lets nearer neighbours count for more. Despite their simplicity, students often struggle to see *why* a single close neighbour can outvote several distant ones, or how the metric, k and weighting jointly alter a prediction. Static figures show the outcome but not the mechanism.

People understand algorithms faster when they can play against them. We therefore turn the comparison into a scored three-way contest, *You* versus *kNN* versus *wkNN*, and expose every internal quantity of the classifiers. The objectives of this work are to:

- visualise kNN and wkNN side by side on identical data;
- let the learner compete against both algorithms with a live scoreboard;
- expose neighbour rank, distance, weight and vote share;
- teach edge cases (ties, $k > n$, coincident points, metric geometry) through puzzles;
- run entirely in the browser, with no backend, ML-library installation or accounts.

The scope is two-dimensional data with up to four classes, three metrics, four weighting schemes, four datasets and seven puzzle levels. High-dimensional data, parametric training and server-side persistence are out of scope.

The remainder of the paper is organised as follows. Section II describes the system and its modes, Section III gives the underlying mathematics, Section IV details the datasets and puzzle design, Section V covers implementation and testing, Section VI discusses results, novelty and limitations, and Section VII concludes.

II. System Overview

A. Application Modes

KNN Arena offers three modes.

1) **Play Mode.** A mystery star with a hidden class appears. The player guesses via class buttons or keys 1–4; the truth and both models' votes are then revealed and the scoreboard updates. An optional *Hard Mode* draws mystery points from contested or low-margin test points where kNN and wkNN disagree.
2) **Sandbox Mode.** The player clicks to probe any location, sees neighbour rays, the metric ball and both predictions, and may add custom training points that update the decision regions live.
3) **Wedge Puzzles.** Seven levels in which a limited number of points must be placed so that kNN and wkNN produce a prescribed pair of different classes.

B. Visualisation and Inspection

The arena canvas draws neighbour rays whose thickness is proportional to the wkNN weight, and a dashed *metric ball* at the k-th neighbour distance whose shape (circle, diamond or square) follows the selected metric (Fig. 3). Decision-region heatmaps can show the kNN regions, the wkNN regions, or a



Fig. 1. Main interface of KNN Arena.

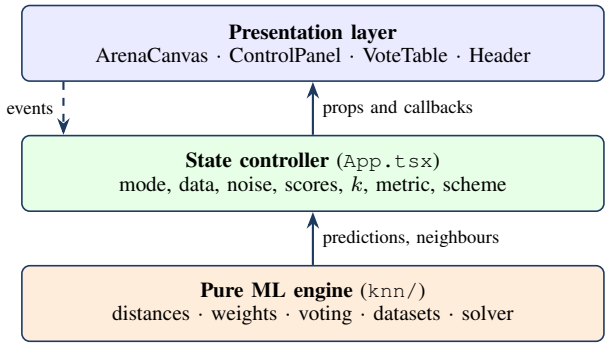


Fig. 2. Layered architecture of KNN Arena.

TABLE I
TECHNOLOGY STACK

Layer	Technology
Framework	React 18 (Hooks)
Language	TypeScript 5.7
Bundler	Vite 6
Styling	Tailwind CSS 3
Rendering	HTML5 Canvas 2D, ResizeObserver
Testing	Vitest
Animation / icons	canvas-confetti, lucide-react
Original prototype	Python 3, NumPy, Matplotlib, Tkinter

disagreement map highlighting only where the two differ. User-added points are drawn as diamonds and noisy-label points are faded. Uniform letterbox scaling keeps circles circular on any window size. A live *Vote Table* lists rank, distance, weight, share (%), class and point origin. The configuration panel provides a k slider, three metrics, four weighting schemes, four datasets and label noise from 0 to 100%.

C. Architecture

The system has three layers (Fig. 2): a pure TypeScript ML engine with no React dependency, a React state controller (`App.tsx`) acting as the single source of truth, and presentational components rendering to HTML5 Canvas and the DOM. Hooks (`useState`, `useMemo`, `useCallback`) are sufficient; no external state library is required. Table I lists the technology stack.

TABLE II
DISTANCE METRICS

Metric	Formula	Ball
Euclidean (L_2)	$\sqrt{\Delta x^2 + \Delta y^2}$	Circle
Manhattan (L_1)	$\lvert\Delta x\rvert + \lvert\Delta y\rvert$	Diamond
Chebyshev (L_∞)	$\max(\lvert\Delta x\rvert, \lvert\Delta y\rvert)$	Square

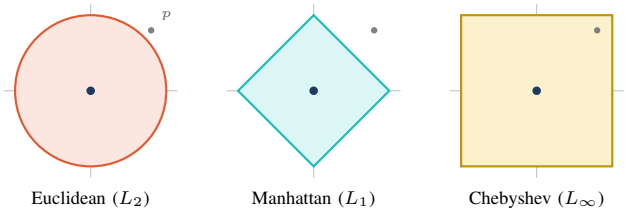


Fig. 3. Metric balls of radius 1 around a query (dot). The same point $p = (0.8, 0.8)$ lies outside the Euclidean and Manhattan balls but inside the Chebyshev ball.

III. Algorithms and Mathematics

A. kNN and Weighted kNN

For a query $\mathbf{q}$, let $\mathcal{N}_k(\mathbf{q})$ be its k nearest training points with labels y_i and distances d_i. Standard kNN gives each neighbour one vote:

$$\hat{y}_{\text{kNN}} = \arg\max_c \sum_{i=1}^{k} \mathbf{1}[y_i = c]. \tag{1}$$

Weighted kNN scales each vote by a distance-dependent weight w_i:

$$\hat{y}_{\text{wkNN}} = \arg\max_c \sum_{i=1}^{k} w_i \, \mathbf{1}[y_i = c]. \tag{2}$$

Both predictors share one sorted neighbour list, so a single $O(n)$ distance pass followed by a partial sort ($O(n \log k)$) serves both.

B. Tie-Breaking

When classes tie, the class of the single nearest neighbour wins. This is implemented by adding $\varepsilon = 10^{-9}$ to that class's score before the $\arg\max$, ensuring deterministic behaviour that matches between the Python and TypeScript versions.

C. Distance Metrics

Three Minkowski-family metrics are supported (Table II, Fig. 3). The ball shape matters pedagogically: under Chebyshev distance, diagonal points can be unexpectedly close.

D. Weighting Schemes

Four schemes are provided (Table III, Fig. 4); Uniform reproduces standard kNN. The Gaussian scheme uses an *adaptive bandwidth* $h = d_k + \varepsilon$, the distance to the k-th neighbour, so h is small in dense regions and large in sparse ones. Consequently, removing the k-th neighbour shrinks h and changes *every* weight, a property exploited by Level 7.

TABLE III
WEIGHTING SCHEMES (ε IS A SMALL CONSTANT)

Scheme	Weight w_i
Uniform	1
Inverse (default)	$1/(d_i+\varepsilon)$
Inverse squared	$1/(d_i^2+\varepsilon)$
Gaussian (adaptive)	$\exp\!\left(-3\,(d_i/h)^2\right),\ h=d_k+\varepsilon$

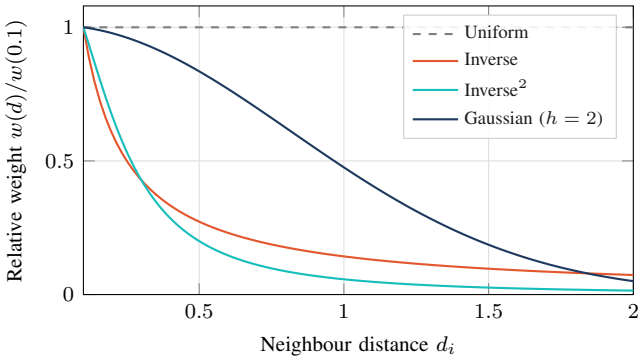


Fig. 4. Relative weight versus distance for the four schemes ($\varepsilon = 0.05$ for illustration; weights normalised to 1 at $d = 0.1$).

TABLE IV
BUILT-IN DATASETS

Dataset	Classes	Points	Teaching point
Blobs	3	240	Easy, near-linear
Moons	2	240	Small k follows curves
Rings	2	240	Radial boundary
Spirals	3	330	Needs small k

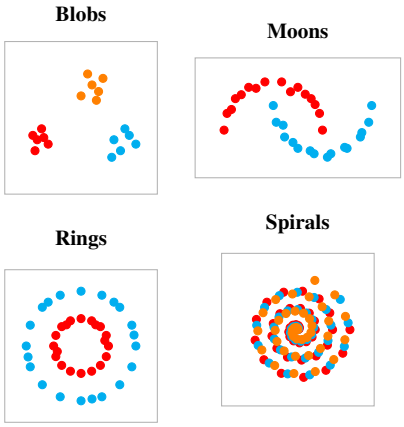


Fig. 5. Illustrative samples of the four built-in datasets, drawn with the generator parameters of Table IV (subsampled for clarity). Colours follow the app's class palette.

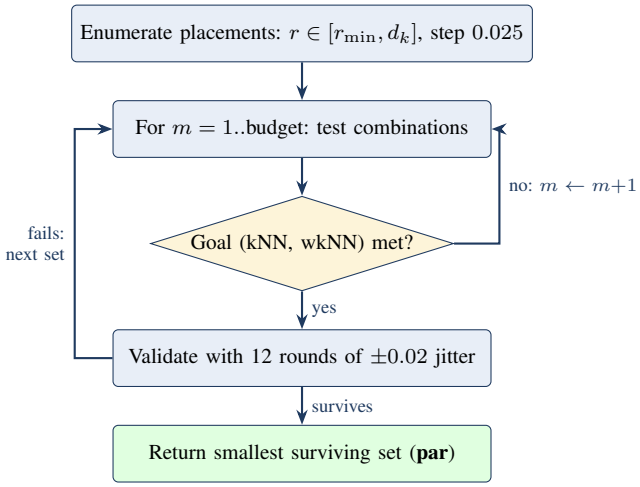


Fig. 6. Flow of the jitter-resilient par solver.

IV. DATASETS AND PUZZLE DESIGN

A. Procedural Datasets

All datasets are generated by a seeded linear-congruential RNG, so every run is reproducible (Table IV, Fig. 5). Each dataset carries a distinct teaching point: Blobs for near-linear separation, Moons and Spirals for the effect of k on curved structure, and Rings for radial non-linear boundaries. Data are split 80/20 into training and test pools.

B. Stable Label-Noise Model

Persistent per-point random vectors $u_i \in [0, 1)$ and class shifts s_i are stored once. A label is flipped when $u_i < p/100$ for noise level p:

$$y_i' = \begin{cases} (y_i + s_i) \bmod C, & u_i < p/100, \\ y_i, & \text{otherwise.} \end{cases} \tag{3}$$

Because the vectors are stored, raising and lowering the slider reveals and restores exactly the same points.

C. Wedge Puzzles

Each Wedge level supplies fixed training points around an origin star at $(0, 0)$, a placement budget, a no-go radius $r_{\min}$ and a goal pair (kNN class, wkNN class) to be achieved simultaneously. The objective is therefore not to classify correctly, but to *engineer a disagreement*. Scoring is three stars for a solution within par, two for par+1, and one for any solution within budget. Table V summarises the seven levels.

D. Jitter-Resilient Par Solver

Par values and hints come from an automated solver (Fig. 6). It enumerates candidate placements on a distance grid from $r_{\min}$ to d_k in steps of 0.025; for $m = 1, \ldots,$ budget it tests every m-combination against the goal; each candidate solution is then validated under 12 rounds of ± 0.02 positional jitter, and the smallest surviving set is returned as par. This guarantees that hinted solutions are reproducible with a mouse and not artefacts of floating-point precision.

V. IMPLEMENTATION AND TESTING

A. Rendering

The canvas renderer draws, from bottom to top: a 110×110 decision heatmap, training points, neighbour rays, the metric

TABLE V
WEDGE PUZZLE LEVELS

#	Name	k	Metric	Scheme	Budget / $r_{\min}$	Goal (kNN / wkNN)
1	Close Call	7	Euclidean	Inverse	2 / 0.20	Fire / Water
2	Crowd Control	7	Euclidean	Inverse	3 / 0.20	Water / Fire
3	Swap Places	7	Euclidean	Inverse	3 / 0.20	Water / Fire
4	Sealed Zone	9	Euclidean	Inverse^2	3 / 0.45	Fire / Water
5	Diamond Rush	5	Manhattan	Inverse	3 / 0.30	Fire / Water
6	Boxed In	5	Chebyshev	Inverse	2 / 0.25	Fire / Water
7	Bandwidth Squeeze	7	Euclidean	Gaussian	2 / 0.35	Water / Fire

TABLE VI
AUTOMATED TEST SUITE (19 TESTS, ALL PASSING)

File	Tests	Coverage
`engine.test.ts`	8	Metric formulas, `ballPoly`, uniform/inverse/Gaussian weights, tie-breaking
`levels.test.ts`	6	Level validity, par solution, ± 0.02 jitter (12 trials), $k = 1$, $k > n$, coincident points
`components.test.tsx`	5	Header, Play controls, Wedge telemetry, Sandbox clear, Vote Table states
Total	**19**	

TABLE VII
KNN ARENA VERSUS TYPICAL STATIC DEMONSTRATIONS

Aspect	Static demo	KNN Arena
Interaction	View-only or single probe	Play, probe, add points, solve puzzles
Algorithms	Usually kNN only	kNN and wkNN side by side
Learner role	Observer	Competitor
Internals	Rarely visible	Full neighbour table and weights
Edge cases	Not covered	Ties, $k > n$, duplicates, metrics
Installation	Often an ML stack	Static web page

ball, the no-go ring, ghost hint placements, the focus star, and the title and legend. The world-to-pixel transform uses a single uniform scale

$$s = \min\left(\frac{W}{x_{\text{span}}}, \frac{H}{y_{\text{span}}}\right), \tag{4}$$

$$p_x = o_x + (x - x_{\text{mid}})s, \qquad p_y = o_y - (y - y_{\text{mid}})s, \tag{5}$$

with the y-axis flipped. The uniform scale keeps Euclidean, Manhattan and Chebyshev balls geometrically faithful at any window size.

B. From Prototype to Web Application

The project began as a Python/Tkinter prototype (`KNN.py`) using NumPy and Matplotlib, and was re-implemented in React 18 and TypeScript. The module mapping is one-to-one (for example, `KNN.predict_from_neighbors()` → `predictFromNeighbors()`, `solve_wedge()` → `solveWedge()`, and `np.random.default_rng()` → a seeded LCG). Parity was verified for the par-solver output of all seven levels, tie-breaking, the `noiseU` comparison and the Gaussian bandwidth $h = d_k + \varepsilon$.

C. Automated Testing

Nineteen Vitest tests were written across three files (Table VI), and all pass. The production build is a static site (HTML ≈1.0 kB, CSS ≈20 kB, JavaScript ≈228 kB, or 72 kB gzipped) that can be hosted on any static server without a backend.

VI. RESULTS AND DISCUSSION

A. Original Contributions

The key contributions of this work are:

1) a head-to-head game format in which the human, kNN and wkNN classify the same point;
2) Wedge Puzzles whose goal is to engineer kNN/wkNN disagreement;
3) a jitter-resilient automated par solver;
4) a disagreement heatmap that shows only the regions where the two algorithms differ;
5) hard-mode sampling from contested or low-margin regions;
6) a stable, reversible label-noise model;
7) the *Bandwidth Squeeze* mechanic exploiting the adaptive Gaussian bandwidth;
8) geometry-faithful rendering of metric balls;
9) a library-free implementation in both Python and TypeScript.

Table VII contrasts KNN Arena with typical static demonstrations.

B. Experimental Observations

Table VIII records Play-mode accuracy of the player, kNN and wkNN under the default configurations.

C. Educational Use

The tool supports classroom demonstrations (same query under kNN and wkNN), lab exercises (finding the best k on Moons versus Spirals), homework (completing all seven levels

TABLE VIII
PLAY-MODE ACCURACY (%) BY CONFIGURATION

Dataset	k	Metric	Scheme	You	kNN	wkNN
Moons	7	Euclid.	Inverse	TODO	TODO	TODO
Rings	7	Euclid.	Inverse	TODO	TODO	TODO
Blobs	7	Euclid.	Inverse	TODO	TODO	TODO
Spirals	3	Euclid.	Gaussian	TODO	TODO	TODO

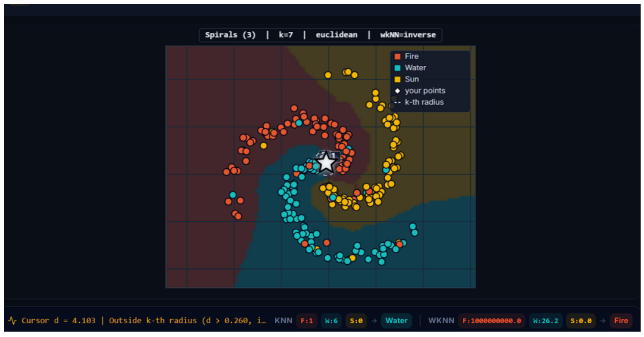


Fig. 7. Regions where kNN and wkNN disagree (Spirals).

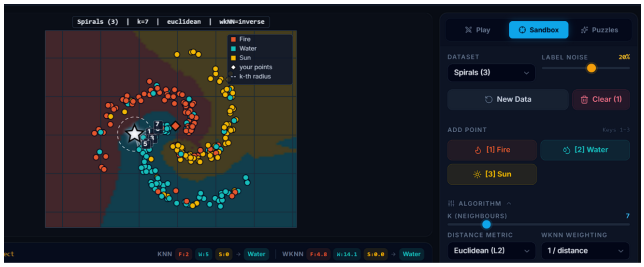


Fig. 8. Label-noise sensitivity of the decision regions under 20% label noise.

at three stars with explanations), self-study with outliers and noise in the Sandbox, and friendly competition against both models.

D. Limitations

The current version supports mouse input only (no touch), and the 110×110 heatmap (12 100 cells) may lag on slow CPUs at high k. Only four built-in datasets are available with no CSV upload, a maximum of four classes, 2-D data only, and single-player use.

E. Future Work

High-priority extensions are custom CSV/JSON dataset upload, touch and long-press support, a per-class confusion matrix, and a cross-validation mode with a best-k finder. Medium-priority items include a k-d/ball-tree index, animated boundary transitions, shareable puzzle solutions via URL, additional kernels and metrics (Minkowski p, cosine, Mahalanobis), and a level editor. Lower-priority items include an accuracy-versus-k curve, a leaderboard, classroom multiplayer, accessibility palettes, an onboarding tutorial and PNG/PDF export.

VII. CONCLUSION

KNN Arena turns the difference between kNN and wkNN into something learners can see, test and compete against. Its original game design, jitter-resilient puzzle solver and from-scratch engine make it both an effective teaching tool and a compact example of clean ML-plus-UI engineering. The roadmap above outlines clear paths towards a broader interactive machine-learning laboratory.

ACKNOWLEDGMENT

The authors thank their guide, Anuradha Banerjee, and the Department of Computer Application, Kalyani Government Engineering College, for their support and guidance.

REFERENCES

[1] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.
[2] S. A. Dudani, "The distance-weighted k-nearest-neighbor rule," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-6, no. 4, pp. 325–327, Apr. 1976.
[3] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.
[4] Meta Platforms, "React documentation." [Online]. Available: https://react.dev
[5] "Vite documentation." [Online]. Available: https://vitejs.dev
[6] "Vitest documentation." [Online]. Available: https://vitest.dev
[7] Mozilla, "Canvas API," MDN Web Docs. [Online]. Available: https://developer.mozilla.org